

UniPINN

Project Architecture Reference

Abstract

UniPINN is a flat-layout Python package for solving PDEs with Physics-Informed Neural Networks. This document describes the package structure, every public module, and the experiment runner workflow.

1 Top-level layout

<code>unipinn/</code>	installable package (<code>pip install -e .</code>)
<code>experiments/</code>	runnable scripts (not part of the package)
<code>scripts/</code>	CLI utilities
<code>tests/</code>	pytest test suite
<code>data/</code>	static data files (meshes, etc.)
<code>doc/</code>	this document and diagrams
<code>.tmp_results/</code>	experiment outputs (gitignored)
<code>pyproject.toml</code>	setuptools build config

The package uses a **flat layout** (`unipinn/` sits at the project root, not inside `src/`) and is *not* published to PyPI. Dependencies are declared in `pyproject.toml` with `setuptools` as the build backend.

2 Package modules

2.1 core — Training engine

Symbol	Purpose
<code>Trainer</code>	Main training loop: <code>fit()</code> , optimizer step, warmup, scheduler
<code>Callback</code>	Base class for hooks: <code>on_epoch_end()</code> , <code>on_train_begin()</code> , etc.
<code>CallbackManager</code>	Ordered list of callbacks, dispatches events
<i>Built-in callbacks (core/callbacks/)</i>	
<code>ConsoleLogger</code>	Prints loss every k epochs
<code>ConsoleLoggerTimeit</code>	Logger + per-epoch wall-clock timing
<code>TimingSummaryCallback</code>	End-of-training timing report
<code>DynamicSwitchCallback</code>	Switches optimizer mid-training (Adam→LBFGS)

2.2 nn — Neural network

Symbol	Purpose
<code>SimpleNN</code>	Fully-connected network with per-layer activation dispatch via <code>arch_config</code> dict
<i>Activations (nn/activations.py) — 10 custom functions</i>	
<code>AdaptiveTanh</code>	Learnable scale on $\tanh(\alpha x)$
<code>DyT</code>	Dynamic tanh variant
<code>SineActivation</code>	$\sin(\omega x)$

SineActivationT, SineActivationT3	Trainable-frequency sine variants
DynSine	Adaptive sine with learnable ω
PolynomialActivationV1/V2	$x + \alpha x^2$ and higher-order polynomial
ChebyshevActivation	Chebyshev polynomial basis
FourierActivation	Fourier series activation

2.3 pde — PDE problems and loss

Symbol	Purpose
PINNLossPoisson1D	$L = w_{\text{pde}} \ -u'' - f \ ^2 + w_{\text{bc}} \ u_{\text{bc}} - g \ ^2$
PINNLossPoisson2D	$L = w_{\text{pde}} \ -\Delta u - f \ ^2 + w_{\text{bc}} \ u_{\text{bc}} - g \ ^2 + w_{\text{data}} \ u - u^* \ ^2$
<i>Benchmarks (pde/benchmarks/)</i>	
BaseBenchmark	ABC with <code>compute_metrics()</code> (11 error metrics)
Poisson1DBenchmarkIndex	Registry: <code>steep_solution</code> , <code>boundary_layer_exp</code> , <code>asymmetric_steep</code> , <code>discontinuous_jump</code>
Poisson2DBenchmarkIndex	Registry: <code>harmonic_rational</code> , <code>steep_product_2d</code>
IrregularDomainBenchmark	File-based sampling for non-rectangular domains
MeshBasedBenchmark	Mesh-based sampling (T3, Q4, Q8, Q9 elements)

2.4 geometry — Mesh and spatial data

Symbol	Purpose
MeshData	Dataclass: nodes, elements, <code>elem_type</code> , <code>elem_areas</code> , boundary groups
ShapeFunctions	Isoparametric mapping and interior sampling for T3/Q4/Q8/Q9
<i>Generation (geometry/generation.py)</i>	
<code>plane_rectangle_mesh()</code>	Structured quad mesh on a rectangle
<code>plane_delaunay_mesh()</code>	Delaunay triangulation mesh
<code>plane_delaunay_q4_mesh()</code>	Delaunay-based Q4 mesh
<i>Quality metrics (geometry/quality.py) — 11 metrics</i>	
<code>shape_quality()</code> , <code>volume_quality()</code> , <code>condition_number()</code> , <code>mean_ratio()</code> , <code>frobenius_norm()</code> , <code>determinant()</code> , <code>trace()</code> , <code>oddy_metric()</code> , <code>winslow_metric()</code> , <code>volume_shape_quality()</code>	
<i>I/O (geometry/io/)</i>	
<code>read_mesh_file()</code>	Dispatcher: routes <code>.msh</code> , <code>.inp</code> , <code>.vtk</code> to readers
<code>read_msh_to_numpy()</code>	Gmsh reader (Q4/Q8/Q9 via element-type dispatch)
<code>read_inp_to_numpy()</code>	Abaqus <code>.inp</code> reader
<code>read_vtk_to_numpy()</code>	VTK legacy reader
<code>load_mesh()</code>	Custom <code>.mesh</code> format reader
<code>load_collocation_points()</code> , <code>load_evaluation_points()</code> , <code>load_boundary_sample()</code> , <code>load_data_sample()</code>	

2.5 metrics — Error evaluation

Function	Description
<code>rmse()</code>	Root mean squared error
<code>max_error()</code>	L^∞ norm

<code>relative_l2()</code>	Relative L^2 error
<code>relative_l_inf()</code>	Relative L^∞ error
<code>mae()</code>	Mean absolute error
<code>mse()</code>	Mean squared error
<code>pearson_r()</code>	Pearson correlation coefficient
<code>r2_score()</code>	Coefficient of determination R^2
<code>nrmse()</code>	Normalised RMSE
<code>energy_norm()</code>	Energy norm (gradient-weighted)
<code>h1_semi_norm()</code>	H^1 semi-norm

2.6 config — Experiment configuration

Symbol	Purpose
<code>BaseExperimentConfig</code>	Shared fields: seed, device, precision, epochs, log_interval
<code>Poisson1DVanillaPINNConfig</code>	1D-specific: benchmark_name, arch_config, loss weights, optimizer
<code>Poisson2DVanillaPINNConfig</code>	2D-specific: adds n_bc_per_edge, n_data, data_weight
<code>load_json_config()</code>	Load config from JSON file
<code>load_toml_config()</code>	Load config from TOML file

Configuration-as-code pattern: all parameters are Python dataclass instances defined in `registry.py` files, not external YAML/JSON. JSON files can be generated on demand via `scripts/generate_config.py`.

2.7 io — Artifacts and workspace

Symbol	Purpose
<code>SafeJSONEncoder</code>	JSON encoder for numpy/torch types
<code>save_models()</code>	Save model state dicts
<code>save_experiment_artifacts()</code>	Save config, model, predictions, history, logs
<code>get_project_root()</code>	Resolve project root directory
<code>get_results_dir()</code>	Get results path for problem/framework
<code>get_next_test_case_dir()</code>	Auto-increment test_case_ N directories
<code>StreamTee</code>	Capture stdout/stderr to buffer
<code>input_with_timeout()</code>	Timed user input with default fallback

2.8 utils — Tensor utilities

Function	Purpose
<code>gradients()</code>	Compute $\partial y / \partial x$ via <code>torch.autograd.grad</code>
<code>to_tensor()</code>	numpy $\rightarrow$ torch tensor with dtype and device
<code>to_numpy()</code>	torch tensor $\rightarrow$ numpy array
<code>get_param_num()</code>	Count trainable parameters in a model
<code>r2_score()</code>	R^2 score (torch version)
<code>abs_r2_score()</code>	Absolute R^2 score

3 Experiment runner workflow

Each experiment script (`vanilla_pinn.py`) follows the same pipeline, parameterised by a config dataclass and a benchmark registry.

1. **Config selection** — load from `registry.py` (batch mode) or `--config JSON` (single mode).
2. **Data generation** — the benchmark’s `generate()` method produces collocation points x_{col} , source values f_{col} , boundary points x_{bc} with targets, evaluation grid x_{eval} with exact solution u_{eval} , and (for supervised mode) sparse interior data $(x_{\text{data}}, u_{\text{data}})$.
3. **Model construction** — SimpleNN built from `arch_config`, moved to the selected device and precision.
4. **Loss setup** — PINNLossPoisson1D/2D with configurable w_{pde} , w_{bc} , w_{data} .
5. **Optimizer and scheduler** — Adam or LBFGS with warmup + StepLR/CosineAnnealing.
6. **Training** — `Trainer.fit` runs the loop, dispatching callbacks for logging, timing, and optimizer switching.
7. **Evaluation** — model predictions on x_{eval} , scatter plots (Exact vs. PINN vs. Error), error metrics.
8. **Artifacts** — `config.json`, `model_main.pt`, `predictions.npz`, `history.json`, `console_log.txt`, prediction plots saved to `.tmp_results/.../seed_N/`.

4 Batch and parallel execution

Both `vanilla_pinn.py` scripts support batch mode via the `--batch` flag. The runner:

1. Selects configs by name (`--configs`), group (`--group`), or all.
2. Expands multi-seed configs into individual tasks.
3. Assigns GPUs round-robin if `--gpus` is given.
4. Runs tasks sequentially (`-j 1`) or in parallel (`-j N`) using `multiprocessing.Pool` with `spawn` context (CUDA-safe).
5. Writes `BATCH_SUMMARY.md` with timing, success/failure table, speedup factor, and error traces.

5 Pre-defined experiment groups

1D Poisson (`experiments/poisson1d/registry.py`)

Group	Configs	Seeds	Varies
SamplingTest	<code>cfg_0 – 3</code>	1	uniform vs LHS × Adam vs LBFGS
RandomSeedTest	<code>cfg_4 – 5</code>	10	10 seeds × Adam vs LBFGS
SampleNumTest	<code>cfg_6 – 13</code>	1	$n_{\text{colloc}} \in \{101, 201, 401\}$, optimizer, epochs

2D Poisson (experiments/poisson2d/registry.py)

Group	Configs	Seeds	Varies
UnsupervisedColloc	cfg_0 – 4	1	sampling method, loss weights, Adam vs LBFGS
SupervisedRecon	cfg_5 – 7	1	sparse data count (50 / 200 / 500)

6 Benchmark problems

1D Poisson: $-u''(x) = f(x)$ on $[-1, 1]$

Name	Solution	Character
steep_solution	$0.1 \sin(ax) + \tanh(bx)$	steep transition + oscillation
boundary_layer_exp	$0.1 \sin(ax) + e^{(b-x-1)/b}$	exponential boundary layer
asymmetric_steep	$0.1 \sin(ax) + \tanh(b(x+0.1))$	off-centre steep transition
discontinuous_jump	piecewise $\sin/\exp \cdot \sin$	jump discontinuity at $x = 0$

2D Poisson: $-\Delta u(x, y) = f(x, y)$ on $[-1, 1]^2$

Name	Solution	Character
harmonic_rational	$\frac{2(1+y)}{(3+x)^2 + (1+y)^2}$	harmonic ($f = 0$), smooth Dirichlet test
steep_product_2d	$(0.1 \sin \omega_x x + \tanh r_1 x) \sin \omega_y y$	multi-scale: high-freq + steep transition

Default parameters for steep_product_2d: $\omega_x = 4\pi$, $\omega_y = 4\pi$, $r_1 = 10$.